mazewall: restricting application workers on Linux

Leanid Piliptsevich

Independent researcher
Corresponding author: pilleo19@gmail.com

Abstract
An application may need access to the network and filesystem even when one of its components needs neither. Giving every worker the same permissions exposes unnecessary resources when that component processes malicious input. This paper presents mazewall, a library that uses Linux security mechanisms to limit selected workers. A process-wide policy blocks operations that the whole application does not need. Additional worker policies restrict system calls and access to files. A document-processing service provides a running example of how these restrictions are chosen, installed, and checked. The current implementation uses Kotlin and targets the Java Virtual Machine, but the underlying controls belong to Linux and can support other language integrations. The paper explains the implementation, its relationship to earlier syscall-filtering systems, and the limits of protection inside a shared process. Existing demonstrations provide preliminary evidence; they do not establish complete protection against code execution or measured performance benefits.

Keywords
Linux security, syscall filtering, Seccomp-BPF, Landlock, least privilege

1. Statement of the problem in general terms

Consider a Linux service that receives an uploaded document, extracts its text, and sends the result to another service. Its request handler needs a network connection. Its document parser needs to read the uploaded file. Its delivery worker needs to send the result. The parser itself does not need to start programs, open new network connections, or read unrelated files. Nevertheless, those operations may be available to it because it runs inside the same application process.

This becomes dangerous when a document makes the parser do something unintended. For example, an XML document can ask a vulnerable parser to include the contents of a local file. A crafted input can also make vulnerable code contact an attacker-controlled server. Fixing the parser remains necessary. Additional operating-system restrictions can reduce the damage by denying the particular file access or connection, even when the parser attempts it.

A policy for the entire application cannot always express these differences. If the delivery worker legitimately needs networking, a blanket network ban would break the service. mazewall therefore combines restrictions for the whole process with stricter restrictions for selected workers. Here, a worker means an operating-system thread dedicated to a class of application tasks. A policy means a description of permitted or forbidden operating-system operations.

A process contains the application's memory and resources. Threads are execution paths inside that process. Two threads can run different tasks while still sharing data and open files. Giving one thread fewer permissions changes what it can request from Linux; it does not divide the process into separate machines. Keeping this distinction in view explains both the benefit and the limitation of mazewall: different workers can have different rules without paying the architectural cost of moving every component into a separate process.

The research question is practical: how can an application restrict a worker without breaking the runtime that executes it, and what protection does that restriction actually provide? The proposed model assumes that the Linux kernel, runtime, and installation code are trusted. It primarily addresses trusted code processing untrusted input. The current library runs on the Java Virtual Machine (JVM), but the problem and the Linux enforcement mechanisms are not specific to Java.

2. Scientific novelty compared to known works

mazewall builds on established Linux mechanisms. Elasticsearch already provides an example of process-level syscall filtering in a JVM service [1]. The contribution presented here is a library-level combination of process restrictions, dedicated-worker policies, filesystem controls, and installation rules. The document-processing example shows how the developer can assign different permissions within one application while accounting for the runtime and shared process state.

Earlier research also automates policy construction and specializes filters. Confine analyses container applications and dependencies to derive syscall policies [2]. SYSPART finds server workers' serving phases through binary analysis and inserts Seccomp installation code at the corresponding transitions [3]. These systems address how to discover and restrict operations required by an existing workload. mazewall gives the application developer an explicit point at which to assign a policy to selected workers.

SysComb tracks developer-specified execution states and uses eBPF to enforce state-dependent syscall filters without rewriting the application [4]. Its work confirms that different thread states can justify different syscall sets. mazewall uses persistent Linux restrictions on dedicated workers and adds Landlock for filesystem policy. It does not claim to introduce thread-specific filtering or to outperform these systems. Its proposed contribution is the practical composition of these controls behind application-facing interfaces, together with an explicit account of what each boundary protects.

3. Brief presentation of the proposed solution

A system call is a request from a program to the Linux kernel. Opening a file, creating a socket, and executing a program are examples. Linux checks these requests regardless of which source language produced them. mazewall uses two existing Linux facilities: Seccomp-BPF to restrict system calls, and Landlock to restrict access to resources such as files. These controls complement normal file permissions and any restrictions already imposed by the deployment environment.

Seccomp-BPF runs a small filter when a thread makes a system call. The filter can inspect the call number and numeric arguments, then allow the call or reject it. The kernel can return a permission error without performing the requested operation. A filter can distinguish operations such as creating a socket and executing a program. It cannot directly read a pathname stored at a pointer supplied by the application. Linux documentation consequently describes syscall filtering as one part of building a sandbox, rather than a complete sandbox by itself [5].

Landlock supplies the filesystem part of the policy [6]. For the document parser, a policy can permit reading a selected input directory while denying covered accesses elsewhere. The actual protection depends on the rights requested and those supported by the running kernel. Seccomp answers a question such as whether a worker may execute a program; Landlock can answer whether it may open a particular file with a requested kind of access. The library needs both kinds of restriction for the example.

For instance, suppose documents are stored under /srv/inbox and an unrelated credential file is stored under /srv/secrets. The intended parser policy permits the required reads under /srv/inbox, together with files needed by the runtime, but excludes the credential file. An ordinary document can then be opened. If a malicious document makes the parser attempt to open the credential file, Linux checks the attempted access against the installed rules. The decision depends on the resource access, not on recognizing the malicious document's syntax or identifying the vulnerability that triggered it.

The first level is a process-wide baseline: restrictions that every part of the application must obey. If the service never needs to execute another program after startup, that behaviour can be blocked throughout the process. A narrower policy is then installed on the parser workers. It can prevent them from creating network connections and limit which files they can open, while the delivery workers retain the network access needed for their job.

The restrictions accumulate. An operation must survive the outer deployment controls, the process baseline, and the restrictions on the thread making the request. A worker cannot use its own policy to regain an operation already denied by the baseline. Likewise, an application cannot use mazewall to undo a restriction imposed by its container or host. This allows a deployment policy and an application policy to serve different purposes without granting extra permissions.

The running example is therefore concrete: the process must not execute another program; the parser may open allowed input files but must not create a network connection; the delivery worker may connect where its own policy permits. These are intended policy outcomes, not measurements from an experiment. They also concern operations made by the relevant worker. They do not imply that the parser has a separate memory space or that every possible way of communicating has disappeared.

3.1. Recording intended behaviour: SBoB and policies

The developer starts by identifying what the workload should do. For the parser, this includes the uploaded document, any required schemas, and runtime files needed while processing it. It excludes unrelated secrets, output directories, and network access. This step records intended behaviour. Observing what an application happens to do during one execution cannot decide which of those actions should be trusted.

This distinction is central to a Software Bill of Behavior (SBoB). A Software Bill of Materials lists what software contains; an SBoB describes what it is intended to do when it runs. The published SBoB specification, version 0.0.3, covers expected file access, child processes, network destinations, and Linux capabilities [7]. It is currently a proposed specification. In the running example, the intended behaviour includes reading uploaded documents and delivering results, but excludes reading credentials or executing another program.

SBoB provides a way to communicate that intent, while mazewall supplies Linux controls for enforcing compatible restrictions. For example, a declaration that no external program should be executed can guide a process-wide execution ban. Observed behaviour still needs developer review before becoming policy. This relationship does not imply that every SBoB field maps directly to Seccomp or Landlock, or that the current library implements the complete published format.

mazewall includes profiling facilities that help discover the operating-system operations used by a workload [8]. Representative executions can reveal required files and system calls that were not obvious from the application code. The result is a candidate for review. Successful inputs alone are insufficient: malformed documents, missing files, recovery paths, and shutdown can exercise different operations. Dependency and runtime upgrades can change the required set as well.

After review, the developer validates the policy using both allowed and forbidden operations. A valid document should still be processed successfully. An attempted connection from the parser should fail under its network restriction. An attempted read outside the allowed filesystem area should fail under the applicable Landlock rules. A denial is useful only if the intended workload still works; a configuration that prevents every task from completing has not demonstrated useful containment.

Installation must finish before a worker receives untrusted input. In the current library, required Landlock setup happens before Seccomp installation because the final syscall filter can prohibit the setup operations themselves. A setup failure is reported rather than silently treated as a protected worker. Running without protection is a different deployment choice and must be explicit. The application also has to select restrictions compatible with the Linux features available in its environment.

Restrictions remain attached to the operating-system thread. Seccomp filters cannot be removed to make that thread unrestricted again. Consequently, a restricted parser worker should remain dedicated to compatible parsing tasks. It cannot simply return to a general pool where a later task expects broader permissions. Worker creation, successful installation, task admission, and termination are therefore part of the security design, alongside the policy rules themselves.

The application also needs a defined response to denial. If the parser receives a permission error while processing an uploaded document, it can reject that document and report the failure to its caller. Retrying the same operation on an unrestricted worker would defeat the intended policy. During development, the error may instead reveal a missing legitimate requirement. The developer should investigate and revise the policy deliberately, then rerun the allowed and forbidden cases. An unexpected failure is therefore evidence to examine, not an automatic reason to grant broader access.

3.2. Language support

mazewall currently uses Kotlin and the JDK Foreign Function and Memory API to call Linux interfaces. The same Linux controls can support bindings for other languages; those bindings are not yet demonstrated here.

Each integration must preserve operations needed by its runtime and install restrictions on the correct operating-system thread. Logical tasks such as coroutines may share or move between threads, so restrictions do not automatically follow them. The current JVM integration uses dedicated platform threads for installation.

3.3. What the restrictions protect, and what they leave open

The direct protection is straightforward: when a restricted worker requests a forbidden operation, the applicable Linux control denies it. Changing the language-level caller does not remove the installed filter. This can prevent a parser from opening a prohibited file or creating a prohibited connection. However, an application process contains more than the worker's future system calls. Shared memory, already-open resources, and other workers remain important.

First, threads share memory. A thread restriction does not create a private heap or prevent access to objects already available to the code. Second, a file descriptor is a handle to an opened file or socket. Preventing a worker from opening a new socket does not necessarily stop it from writing through an existing one. A policy allowing ordinary writes for legitimate I/O may leave that route available. Resource ownership therefore has to be considered before interpreting a network or filesystem policy as complete isolation.

Third, execution can be delegated. If compromised code can submit a task to a less-restricted worker, the task runs with that worker's restrictions. The process baseline still blocks the operations covered by the baseline, but the original parser policy does not travel with the submitted task. For this reason, dedicated-worker containment is most useful for trusted code handling risky data. Arbitrary hostile code may require a separate process with carefully limited communication and resources.

The repository contains a historical report of eight vulnerable-application scenarios [8]. A protected request intended to make the server contact another endpoint reports a permission error. A malicious XML example no longer returns the tested local-file content. These observations are consistent with the intended restrictions, but several report entries mainly show application errors or missing callbacks. Such symptoms alone do not prove that the vulnerable operation was reached and then denied by the kernel.

The report also includes a database-query attack that succeeds with and without containment. This illustrates a useful limit: harmful application behaviour can occur entirely through operations that remain permitted. The records are preliminary observations from the existing JVM demonstration, not new experiments for this paper. They do not establish a general exploit-prevention rate, portability across runtimes, or performance overhead.

A stronger evaluation should run identical inputs without containment, with only the process baseline, and with the additional worker policy. Each run should preserve a successful benign control and independently check the intended effect: whether a connection reached its destination, a program executed, or a file changed. The policy, source revision, kernel, runtime version, architecture, and outer deployment restrictions should be recorded. Tests involving an existing

descriptor and delegation to another worker would also check whether observed behaviour matches the stated limitations.

4. Conclusions containing main obtained results

mazewall applies Linux restrictions at two useful levels: a baseline for the application process and additional rules for dedicated workers. Seccomp limits system calls; Landlock adds filesystem access controls. The current Kotlin/JVM implementation demonstrates the integration approach, while the kernel model can guide bindings for other languages. Each binding must still preserve runtime progress and attach restrictions to the correct operating-system thread.

The main design result is a practical set of rules for using these controls together: define the workload's needs, review observations, install protection before accepting tasks, and keep restricted workers dedicated to compatible work. The analysis also explains why shared memory, existing handles, and delegation limit what protection inside one process can promise. Controlled experiments remain necessary to establish effectiveness and cost across workloads.

Declaration on generative AI

OpenAI Codex assisted with source discovery, manuscript drafting, revision, and document preparation. The author is responsible for verifying the technical claims, references, and final text.

References

[1] Elastic, Bootstrap checks, System call filter check. https://www.elastic.co/guide/en/elasticsearch/reference/current/bootstrap-checks.html (accessed 7 September 2026).

[2] S. Ghavamnia, T. Palit, A. Benameur, M. Polychronakis, Confine: Automated System Call Policy Generation for Container Attack Surface Reduction, in: RAID 2020, USENIX Association, 2020, pp. 443-458. https://www.usenix.org/conference/raid2020/presentation/ghavanmnia.

[3] V. L. Rajagopalan, K. Kleftogiorgos, E. Göktaş, J. Xu, G. Portokalidis, SYSPART: Automated Temporal System Call Filtering for Binaries, arXiv:2309.05169v2, 2023. https://arxiv.org/abs/2309.05169.

[4] M. Rossi, M. Abbadini, M. Beretta, D. Facchinetti, S. Paraboschi, SysComb: Fine-Grained Transparent System Call Filtering for Attack Surface Reduction, preprint, arXiv:2608.26871v1, 2026. https://arxiv.org/abs/2608.26871.

[5] The Linux kernel contributors, Seccomp BPF (SECure COMPuting with filters). https://docs.kernel.org/userspace-api/seccomp_filter.html (accessed 7 September 2026).

[6] The Linux kernel contributors, Landlock: unprivileged access control. https://docs.kernel.org/userspace-api/landlock.html (accessed 7 September 2026).

[7] Bill of Behavior, SBOB specification, version 0.0.3, proposed specification, 2026. https://billofbehavior.com/bob/docs/spec/ (accessed 8 September 2026).

[8] L. Piliptsevich, mazewall source repository, implementation guidance, and vulnerable-application demonstration report. https://github.com/Pilleo/mazewall (local checkout inspected 7 September 2026; reported experiments not independently reproduced).